Graphs Basics

SE 2228 Presentation – Analysis & Design of Algorithms

T.A. Hatice Obuz

hatice.obuz@yasar.edu.tr



YAŞAR ÜNİVERSİTESİ

Contents

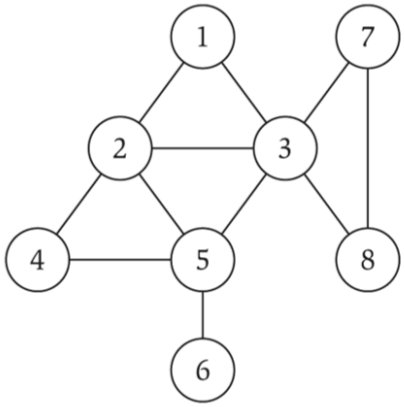
- Graphs Introduction
- Types of Graphs
- Representing Graphs
- Common Graph Theory Problems
- Topological Sort
- Dijkstra's Shortest Path Algorithm
- Prim's Minimum Spanning Tree Algorithm
- Kruskal's Minimum Spanning Tree Algorithm

Graphs Introduction

- A graph denoted by $G = (V, E)$ consists of a set V of vertices and a set E of edges between the vertices.
- Captures pairwise relationship between objects.

Notation: $G = (V, E)$

- V: nodes (vertices)
- E: edges between pairs of nodes.
- n = number of nodes, m = number of edges



$\boldsymbol{V} = \{ 1, 2, 3, 4, 5, 6, 7, 8 \}$
$\boldsymbol{E} = \{ 1–2, 1–3, 2–3, 2–4, 2–5, 3–5, 3–7, 3–8, 4–5, 5–6, 7–8 \}$
$\boldsymbol{m} = 11$, $n = 8$

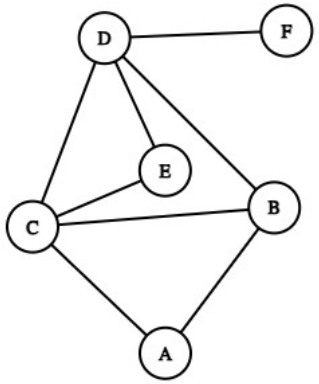
Types of Graphs

- **Undirected Graph**
- **Directed Graph (Digraph)**
- **Weighted Graph**
- **Trees!!!**
- **Directed Acylic Graphs (DAGs)**
- **Bipartite Graphs**
- **Complete Graphs**

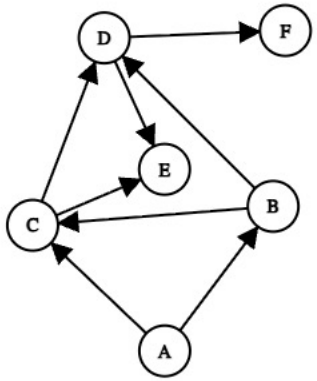
Types of Graphs

Undirected Graph: An **undirected graph** is a graph in which edges have no orientation. The edge (u, v) is identical to the edge (v, u).

Directed Graph (Digraph): A **directed graph** is a graph in which edges have orientations. For example, the edge (u, v) is the edge from node u to node v.



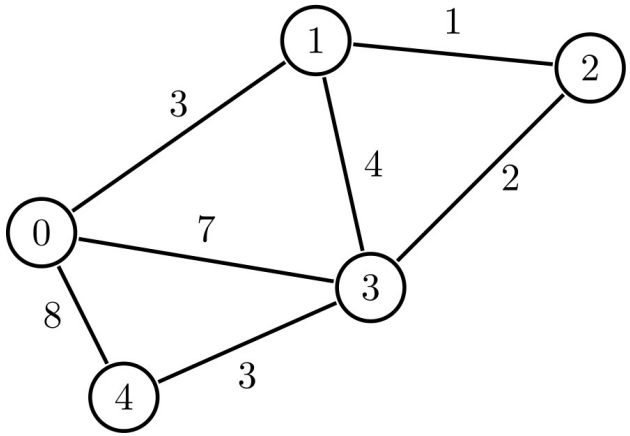
Undirected Graph



Directed Graph

Types of Graphs

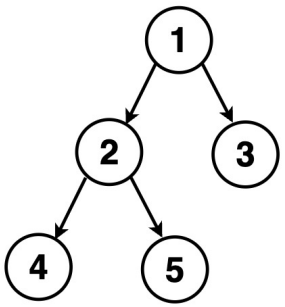
Weighted Graph: Many graphs can have edges that contain a certain weight to represent an arbitrary value such as cost, distance, and quantity.



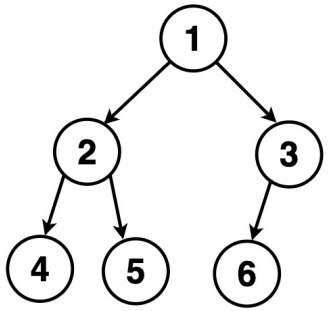
Weighted Graph

Types of Graphs

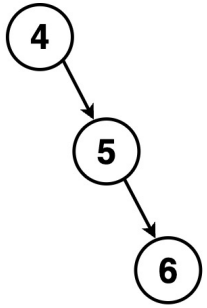
Trees: A tree is an **undirected connected graph** with **no cycles**. Equivalently, it is a connected graph with N nodes and N-1 edges.



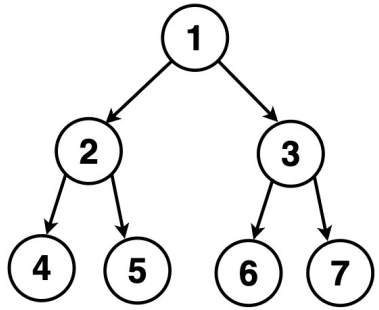
Full Binary Tree



(Almost) Complete Binary Tree



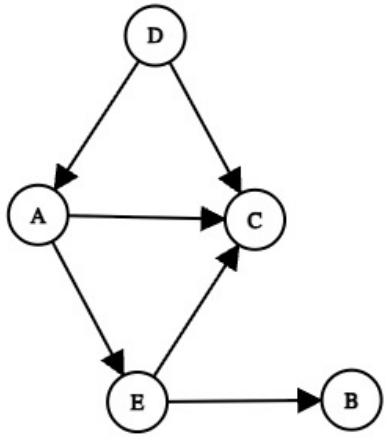
Degenerate Binary Tree



Perfect Binary Tree

Types of Graphs

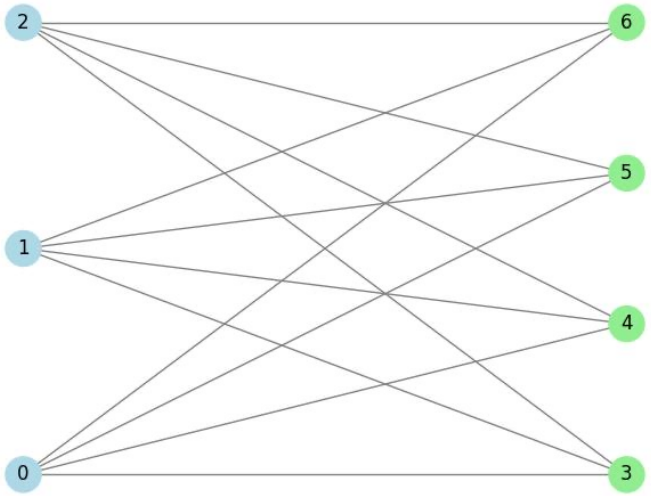
Directed Acylic Graphs (DAGs): DAGs are directed graphs with **no cycles**. These graphs play an important role in representing structures with dependencies.



Directed Acyclic Graph

Types of Graphs

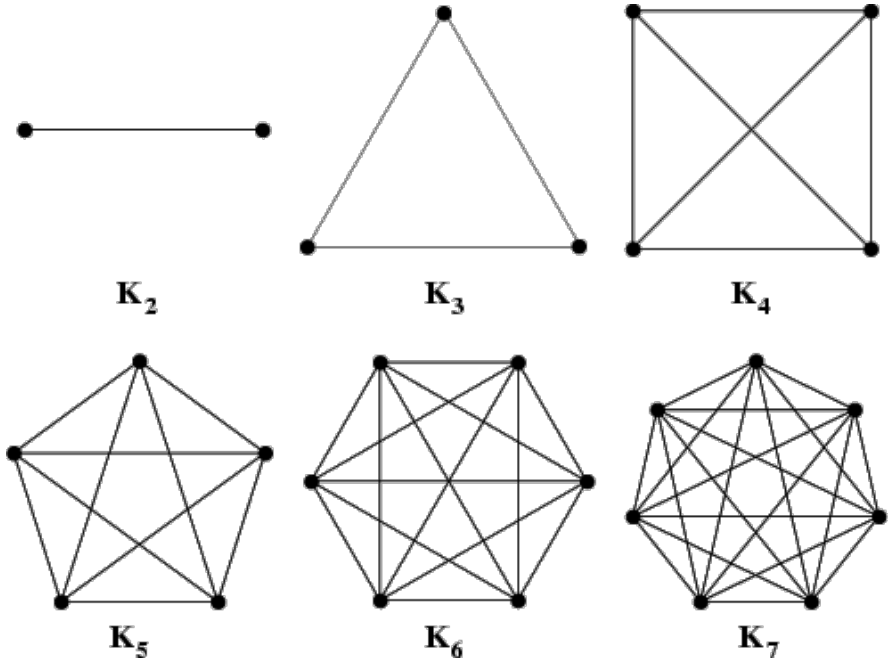
Bipartite Graphs: A bipartite graph is one whose vertices can be split into **two independent groups** U, V such that every edge connects betweens U and V.



Bipartite Graph

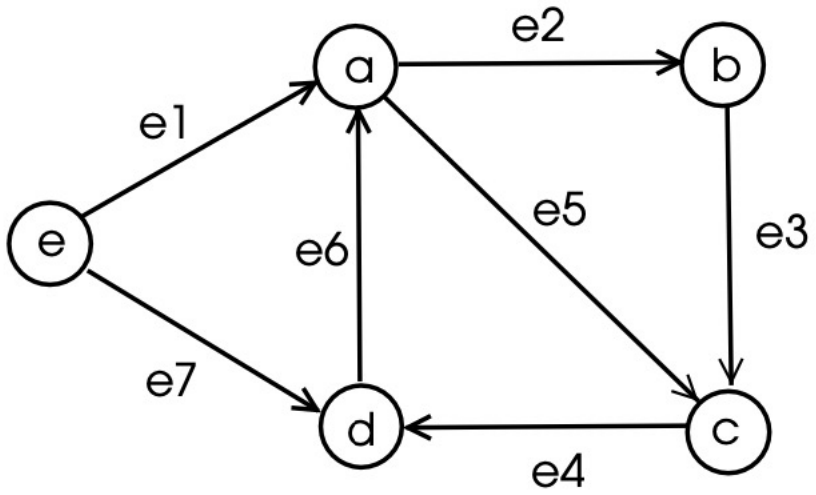
Types of Graphs

Complete Graphs: A complete graph is one where there is a unique edge between every pair of nodes.



Representing Graphs
Adjacency Matrix

- An adjacency matrix m is a very simple way to represent a graph. The idea is that the cell m[i][j] represents the edge weight of going from node i to node j for weighted graphs.
- The cell m[i][j] represents an edge between two nodes for unweighted graphs (write 1 instead of weights).

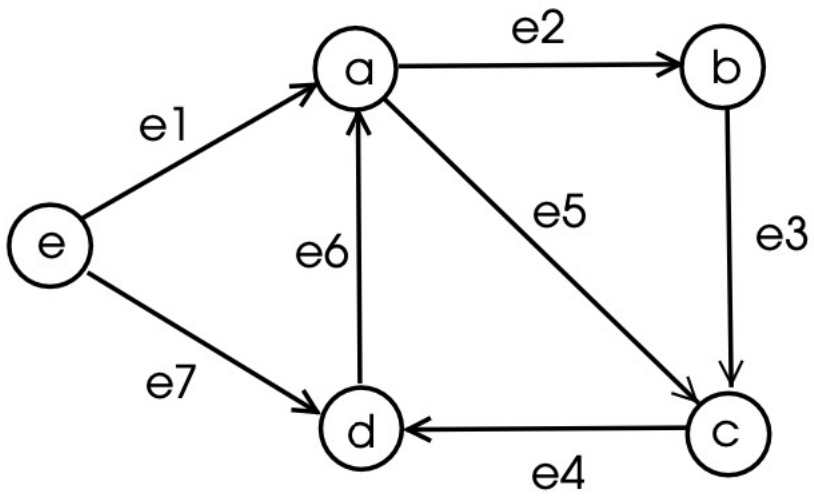


	a	b	c	d	e
a	0	1	1	0	0
b	0	0	1	0	0
c	0	0	0	1	0
d	1	0	0	0	0
e	1	0	0	1	0

Adjacency Matrix

Representing Graphs
Adjacency List

- An **adjacency list** is a way to represent a graph as a map from nodes to lists of edges.




```
graph = {
    a: [b, c],
    b: [c],
    c: [d],
    d: [a],
    e: [a, d]
}
```

Adjacency List

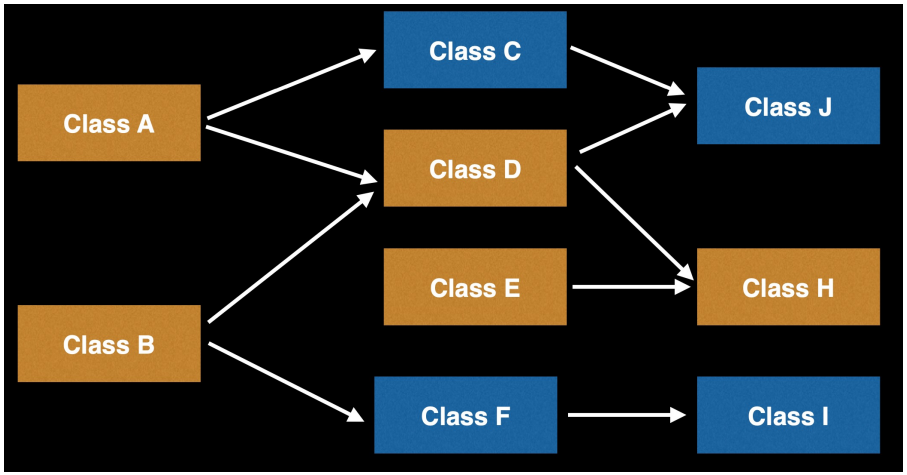
Common Graph Theory Problems



- **Shortest Path:** Given a weighted graph, find the shortest path of edges from node A to node B. (algorithms: Dijkstra's, Bellmann-Ford..)
- **Connectivity:** Does there exist a path between node A and node B?
- **Minimum Spanning Tree (MST):** A minimum spanning tree (MST) is a subset of the edges of a connected, edge-weighted graph that connects all the vertices together, **without any cycles and with the minimum possible total edge weight.** (algorithms: Kruskal's, Prim's..)

Topological Sort

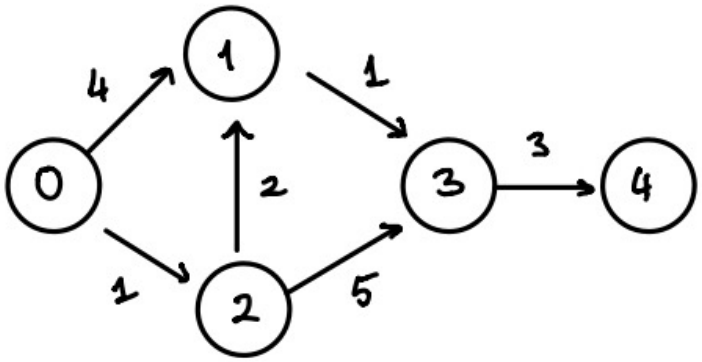
- A **topological ordering** is an ordering of the nodes in a **directed graph** where for each directed edge from node A to node B, node A appears before node B in the ordering.
- **Example**: Suppose you're a student at university X and you want to take **Class H,** then you must take classes **A, B, D and E as prerequisites.** In this sense there is an ordering on the nodes of the graph.



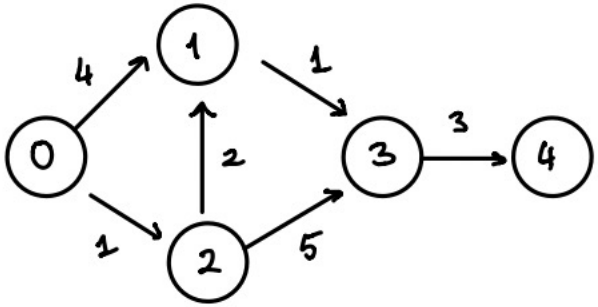
- These orderings are **not unique!!!!**

Dijkstra's Shortest Path Algorithm

- Dijkstra's algorithm is a shortest path algorithm for graphs with non-negative edge weights.
- Let us use Dijkstra's algorithm to find the shortest path of the following graph…



Dijkstra's Shortest Path Algorithm



→ starting node – 0

(index, dist)

key-value pairs

~~(0,0)~~

(1,4)

~~(2,1)~~

~~(1,3)~~

(3,6)

(3,4)

(4,7)

dist

0	1	2	3	4
∞	∞	∞	∞	∞

0	1	2	3	4
0	4	∞	∞	∞

best distance from node 0 to 1 : 4

0	1	2	3	4
0	4	1	∞	∞

best distance from node 0 to 2 : 1

Dijkstra's Shortest Path Algorithm

0	1	2	3	4
0	3	1	∞	∞

best distance from node 2 to 1 : 3

0	1	2	3	4
0	3	1	6	∞

best distance from node 2 to 3 : 6

0	1	2	3	4
0	3	1	4	∞

best distance from node 1 to 3 : 4

0	1	2	3	4
0	3	1	4	7

best distance from node 3 to 4 : 7

Dijkstra's Shortest Path Algorithm

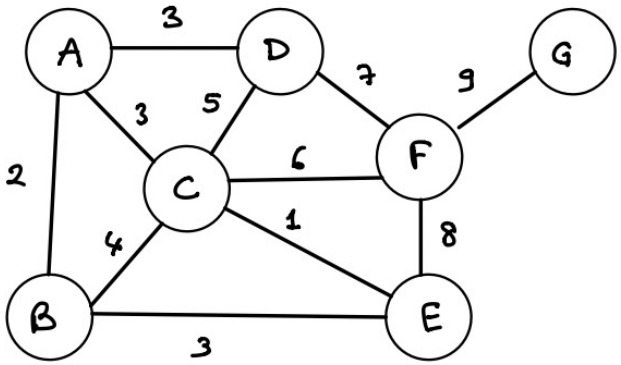
Pseudocode

```
1: Input : G = (V, E, w)                    ▷ connected, weighted graph G and a source vertex s
2: Output : d[n] and p[n]                   ▷ distances and parents of vertices in shortest path tree
3: for all v ∈ V \ {s} do                   ▷ initialize all vertices except source s
4:     d[v] ← ∞
5:     p[v] ← −1
6: end for
7: d[s] = 0; p[s] = s                       ▷ initialize source
8: S ← V
9: while S ≠ Ø do
10:    find u ∈ S with minimum distance value
11:    for all (u, v) ∈ E do                ▷ update neighbor distances to v
12:        if d[v] > d[u] + w(u, v) then
13:            d[v] ← d[v] > d[u] + w(u, v)
14:            p[v] ← u                     ▷ update parent
15:        end if
16:    end for
17:    S ← S \ {u}                          ▷ remove u from searched
18: end while
```

Prim's Minimum Spanning Tree Algorithm

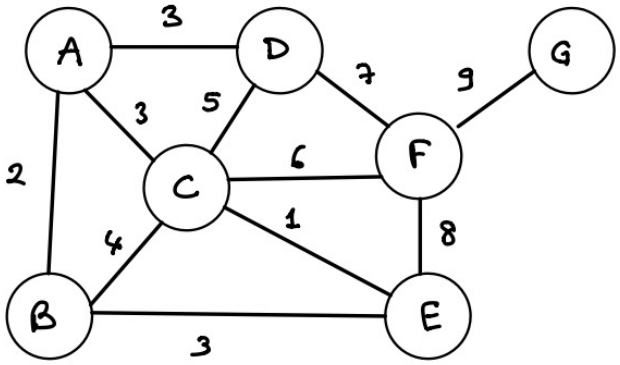
What is a MST?

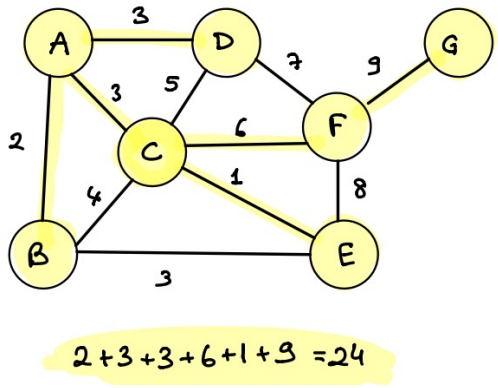
Given an undirected graph with weighted edges, a Minimum Spanning Tree (MST) is a **subset of the edges in the graph which connects all vertices together** (without creating any cycles) while **minimizing the total edge cost.**



Prim's Minimum Spanning Tree Algorithm

- Pick an arbitrary node, for instance A
- Add each node to the visited list when you visit them
- Visited = { } at first





Prim's Minimum Spanning Tree Algorithm

Pseudocode

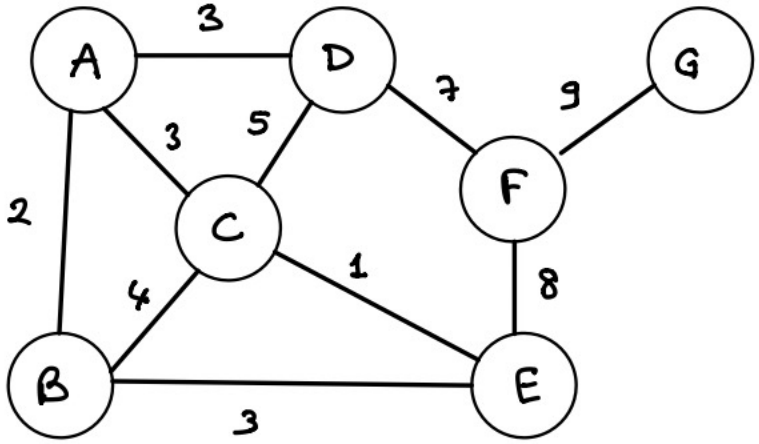
1: **Input** : $G = (V, E, w)$
2: **Output** : MST $T(V_T, E_T)$ of G
3: $V_T \leftarrow \{s\}$
4: $T \leftarrow \emptyset$
5: **while** $V_T \neq V$ **do** ▷ continue until all vertices are visited
6: select the edge (u, v) with minimal weight such that $u \in T$ and $v \in G \setminus T$
7: $V_T \leftarrow V_T \cup \{v\}$
8: $E_T \leftarrow E_T \cup \{(u, v)\}$
9: **end while**

Kruskal's Minimum Spanning Tree Algorithm

YAŞAR ÜNİVERSİTESİ

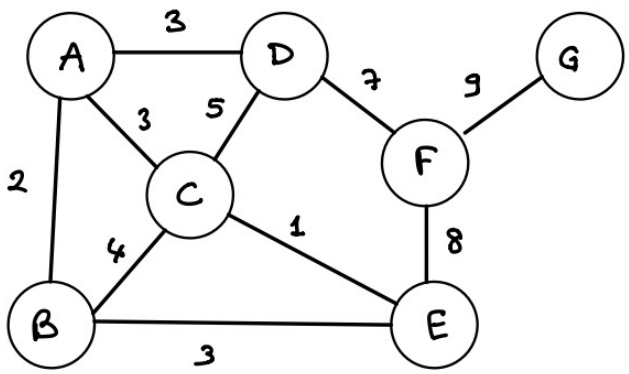
Recall: What is a MST?

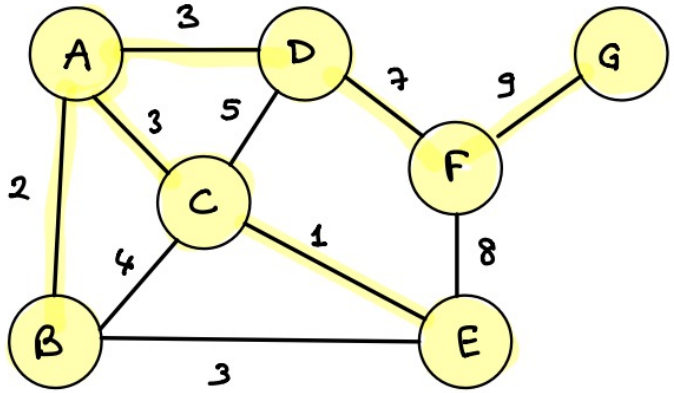
Given an undirected graph with weighted edges, a Minimum Spanning Tree (MST) is a **subset of the edges in the graph which connects all vertices together** (without creating any cycles) while **minimizing the total edge cost.**



Kruskal's Minimum Spanning Tree Algorithm

- Select the edge with minimum weight until there are no cycles formed.





Kruskal's Minimum Spanning Tree Algorithm

Pseudocode

1. **Input**: An undirected weighted graph $G = (V, E, w)$
2. **Output**: An MST $T = (V, E')$ of G
3. $T \leftarrow \emptyset$
4. Sort edges of G in non-increasing order and place them in a queue Q.
5. **repeat**
6. Remove the first edge (u, v) from Q and add it to T if it does not form a cycle with the edges edge that are already included in T.
7. **until** there are $n - 1$ edges in T.



In today's lab..

- Please implement Dijkstra's Algorithm based on the pseudocode given.



Now, it’s your turn to code..

THANK YOU FOR LISTENING!

References

1. C. Daniel, “How to make a weighted graph in Cytoscape.js?,” *Stack Overflow*. https://stackoverflow.com/questions/64038224/how-to-make-a-weighted-graph-in-cytoscape-js
2. “CS Academy.” https://csacademy.com/app/graph_editor/
3. Mquayle, “How to identify six different types of hackers | CMIT Solutions Bothell,” *Bothell, WA 1091 | CMIT Solutions*, Mar. 05, 2025. https://cmitsolutions.com/bothell-wa-1091/blog/different-types-of-hackers/
4. WilliamFiset, “Graph Theory Introduction,” *YouTube*. Mar. 30, 2018. [Online]. Available: https://www.youtube.com/watch?v=eQA-m22wjTQ
5. K. Erciyes, *Algebraic graph algorithms*. 2021. doi: 10.1007/978-3-030-87886-3.
6. “takeuforward - Best Coding Tutorials for Free,” *takeUforward - ~ Strive for Excellence*. https://takeuforward.org/binary-tree/introduction-to-trees/
7. “Graph Theory - Complete bipartite graphs.” https://www.tutorialspoint.com/graph_theory/graph_theory_complete_bipartite_graphs.htm